



CLINICAL PROGRAMMING BOOTCAMP • MODULE 10

Data Quality & Validation

Would this submission survive Pinnacle 21?

You have built seven domains. Now look at them the way a regulator's validation tool does — as one package that must hold together.

Hands-on: Notebook 12 • QC & Validation Checks (SAS)

Pinnacle 21 — what it actually does

It is not magic. It is a library of rules.

Pinnacle 21 (Community and Enterprise) reads your SDTM/ADaM datasets and define.xml, runs thousands of published CDISC conformance rules against them, and produces a report. Sponsors run it before every submission; the FDA runs its own copy when the package arrives.

YOUR DATASETS

`dm.xpt · ae.xpt · ...`

+ RULES

`CDISC conformance library`

= REPORT

`errors · warnings · notices`

The one sentence to remember

A conformance check is a query that should return zero rows. If it returns any, that is a finding. Thousands of rules, one shape. Once you see that, Pinnacle 21 stops being a black box and becomes something you could write yourself — which is exactly what Notebook 12 has you do.

Error, Warning, Notice

ERROR

Blocks the submission

A rule the data **MUST** satisfy. An error means the package is not fit to send until it is fixed or formally explained.

SEX outside {M,F,U} • a missing Required variable • an orphan record

WARNING

Explain or fix

Probably wrong, but the record is still usable. You fix it, or you justify it in the reviewer's guide.

an unexpected severity value • a study day that looks extreme

NOTICE

For your awareness

Informational. Often expected given the study design. You confirm it is intended.

a domain with no records • a permissible variable left null

Severity is a JUDGEMENT about impact, not a fixed label. A sponsor re-classifies findings based on what the study is trying to prove — an unexpected AESEV is a warning in most studies and an error in one whose endpoint is severity-based.

Four questions, asked thousands of ways

KEY INTEGRITY

Is every record uniquely identifiable, and does everything link?

`USUBJID + --SEQ unique · no orphan subjects · SUPP points at a real parent`

REQUIRED DATA

Is everything the standard demands actually present?

`Required variables populated · expected domains exist`

CONTROLLED TERMS

Does every coded value come from its codelist?

`SEX in {M,F,U} · AESER in {Y,N} · units from the UNIT codelist`

VALUE LOGIC

Do the values make clinical and temporal sense?

`no Day 0 · end date >= start date · --BLFL is Y-or-null · result within plausible range`

Notebook 12 writes at least one real check in every category — and the whole ABC-01 library passes all of them, because you built it correctly.

A domain can be perfect and still be wrong

Internal consistency is not correctness.

Every single-domain rule can pass while the SET of datasets is incoherent. These cross-domain checks are the only ones that catch it — and they are the ones a beginner forgets, because each dataset looks fine on its own.

Cross-check	What it compares	If it fails
No orphan subjects	every domain USUBJID vs DM	a subject with data but no demographics
SUPP linkage	SUPPAE.IDVARVAL vs AE.AESEQ	a qualifier pointing at nothing
RFSTDTC agreement	EX first dose vs DM.RFSTDTC	every study day measured from a wrong clock

Why RFSTDTC agreement is the deepest check in the suite

If EX and DM disagree about first dose, every --DY shifts — and shifts CONSISTENTLY, so no within-domain check notices. The error is only visible by comparing two independent records of the same fact. That is the whole reason a submission is validated as a package, never one file at a time.

What a findings table looks like — and how to read it

Result	Severity	Rule	Finding	Rows
FAIL	ERROR	SD0002	ARMCD is Required and must be populated	1
FAIL	ERROR	SD0001	AESTDY must never be 0 - there is no Day 0	1
FAIL	ERROR	SD0011	SUPPAE row with no matching AE parent	1
FAIL	ERROR	SD0004	VSBLFL must be Y or null - never N	1
PASS	ERROR	SD0012	EX first dose disagrees with DM.RFSTDTC	0
PASS	WARNING	CT0002	AESSEV outside expected severity codelist	0

This is the injected-defect run from Notebook 12 — four planted errors, all caught.

How a reviewer reads it

Errors first, worst domain first. Each finding names a RULE (traceable to CDISC), a COUNT, and enough to locate it. Zero rows is a pass — an empty report is the goal.

What you must NOT do

Do not "fix" a finding by deleting the record or blanking the value. A finding is a QUESTION about the data. You resolve it by correcting the source or explaining it — never by hiding it.

The Reviewer's Guide (cSDRG)

Not every finding is a defect. Some are consequences of how the study was designed.

ABC-01 would raise a NOTICE that only 4 of 8 subjects appear in LB. That is not an error — this protocol collects labs on a subset. But the validation tool cannot know that, so it flags it, and YOU must tell the reviewer it is intended.

That explanation lives in the cSDRG — the clinical Study Data Reviewer's Guide

What it is

A document submitted alongside the datasets that explains the study's data to a reviewer.

What it holds

Design decisions, known validation findings and why each is acceptable, deviations from the standard.

Why it matters

It turns a flagged-but-intended finding from a red mark into a documented, defensible choice.

A submission is datasets PLUS documentation. A clean-but-unexplained package and a flagged-but-fully-explained one — the reviewer trusts the second.

Five ways QC goes wrong

1

Writing the check for the GOOD state

A conformance check finds VIOLATIONS. The WHERE describes what must never happen.

→ *expect zero rows*

2

Only checking one domain at a time

The worst errors live BETWEEN datasets, not inside them.

→ *compare EX to DM, SUPP to parent*

3

Treating a clean report as correct

It proves conformance, not truth. Both can be wrong together.

→ *conformance is not correctness*

4

Hiding a finding

Deleting the record makes the report clean and the submission wrong.

→ *fix the source or explain it*

5

Assuming every finding blocks

Severity is a judgement. A WARNING is explained, not necessarily fixed.

→ *read severity, use the cSDRG*